

Projet fil rouge — Audit d'un SaaS de production

Architecture IA · Année 5 NoCode / LowCode & IA · Épreuve finale

1. La mission

Vous êtes une équipe de **développeurs freelances**. Un client vous confie **eRezo** — un SaaS de gestion de contacts et d'événements professionnels, entièrement conçu en *vibecoding* (développement piloté par IA). Le client vous pose trois questions simples :

« Est-ce que mon application est bien construite ? Est-ce qu'un autre développeur pourrait la reprendre ? Est-ce qu'elle est sûre ? »

Votre travail : **auditer eRezo comme un professionnel**, puis remettre un **rapport d'audit** avec des recommandations priorisées. Vous ne développez rien : vous **analysez, testez et jugez** — c'est exactement le rôle d'architecte & reviewer travaillé pendant tout le module.

Le fil conducteur de votre note : appliquez et nommez explicitement les concepts vus en cours (DDD, architecture hexagonale, TDD, pyramide de tests, infrastructure, OWASP). Un audit qui « sent » les bons réflexes mais ne les nomme pas vaut moins qu'un audit qui structure son analyse autour de ces notions.

2. Accès au projet

Ressource	Accès
Code source (lecture seule)	<i>communiqué en séance 4 par le formateur</i>
Application en production	https://erezo.fr
Compte de test	<i>fourni en séance 4</i>

⚠ **La production n'est testable qu'en lecture seule et sous conditions.** Voir le document « **Protocole de pentest & autorisation** » — il est obligatoire, et le signer fait partie de l'exercice.

3. Les 5 axes de l'audit

Votre rapport couvre **cinq axes**. Pour chacun, on attend un constat argumenté + des recommandations.

Axe 1 — Architecture & conception

- Quels sont les **domaines métier** (bounded contexts) de l'application ? Sont-ils bien délimités ?
- Reconnaît-on une **architecture hexagonale** (ports & adapters) ? Une séparation domaine / infrastructure ?
- Le code parle-t-il le langage du métier (**ubiquitous language**) ?
- Où est le **couplage** fort ? Qu'est-ce qui serait difficile à faire évoluer ?

Axe 2 — Tests & qualité

- Y a-t-il des tests ? De quels types (**unitaires / intégration / E2E**) ?
- La **pyramide des tests** est-elle respectée, ou inversée ?
- Voit-on des traces d'une démarche **TDD** ? La couverture est-elle crédible sur le cœur métier ?
- Y a-t-il une **CI** ? Des conventions ?

Axe 3 — Stack technique

- Quelle est la stack réellement utilisée (langage, framework, base de données, services externes, hébergement) ? **Vous devez la découvrir**, elle n'est pas donnée.

- Les choix sont-ils **cohérents** avec un SaaS de cette nature ? Que feriez-vous différemment ?

Axe 4 — Sécurité (pentest IA, OWASP Top 10)

- Conduisez un **audit de sécurité automatisé piloté par IA** sur la production (lecture seule).
- Cherchez les grandes classes de l'**OWASP Top 10** : contrôle d'accès (IDOR), exposition de données, en-têtes de sécurité, injection, configuration.
- **Priorisez** chaque faille : **P0** (critique/immédiat) · **P1** (important) · **P2** (amélioration).

Axe 5 — Reprenabilité & documentation

- Un développeur externe peut-il **comprendre et lancer le projet en moins de 30 minutes** ? (« test du freelance »)
- Y a-t-il un **CLAUDE.md**, un README, une doc d'onboarding ?
- Que manque-t-il pour rendre le projet réellement reprenable ?

4. Livrables & jalons

Quand	Livable
Séance 4	Cadrage, audit statique (archi / tests / stack) démarré, trame de rapport remplie en partie
Séance 5 (en classe)	Pentest sécurité réalisé EN SÉANCE , audit croisé avec un autre groupe, soutenance orale
Vendredi 17 juillet 2026	Rapport d'audit écrit final rendu (voir la trame fournie)

⚠ **Le pentest se fait UNIQUEMENT en séance, en classe.** Aucun test sur eRezo n'est autorisé en dehors du temps de classe. Le rapport rendu le 17 juillet reprend et met au propre **le travail réalisé en cours** — il ne s'appuie sur aucun test mené à la maison.

Format du rendu écrit : un document (PDF) suivant la **trame de rapport d'audit** fournie, à remettre le **vendredi 17 juillet 2026**. Anonymisez toute donnée personnelle réelle éventuellement croisée pendant les tests.

5. Comment vous êtes évalués (/20)

Volet	Points	Ce qu'on regarde
Démonstration	5	Vous reproduisez en direct un finding (une faille, ou une faiblesse d'archi expliquée dans le code)
Présentation des choix techniques	8	Votre analyse archi / stack est juste et argumentée , vous mobilisez le vocabulaire du cours (DDD, hexagonal, TDD, infra), vous défendez vos conclusions face aux questions
Audit sécurité & qualité	7	Rapport de pentest sérieux, failles priorisées P0/P1/P2 , recommandations actionnables, audit croisé intégré
Bonus reprenabilité	+2	Verdict « test du freelance » argumenté (reprenable en < 30 min ? preuve à l'appui)

Un audit **incomplet mais lucide et bien nommé** vaut mieux qu'un audit exhaustif mais confus. On note votre **raisonnement d'architecte**, pas votre exhaustivité.

6. Méthode : l'IA comme copilote d'audit

- Utilisez vos assistants IA pour **accélérer** la lecture de code, le mapping d'architecture, la recherche de vulnérabilités — mais **vérifiez toujours**. Une IA qui « invente » une faille inexistante, c'est un finding faux qui vous coûte des points.
- Documentez **comment** vous avez conduit l'audit (prompts, agents, outils) : la méthode fait partie de la compétence.
- Pour la sécurité : **jamais de test sans cadre**. Vous ne lancez le pentest qu'après avoir lu et signé le protocole.

Bon audit. 🔍